HW3 Readme

Vinisha Bala Dhayanidhi (worked individually)

1. FILES
hw3_topcfg.sh / hw3_topcfg.py - PCFG induction (Part 1)
hw3_parser.sh / hw3_parser.py - baseline PCKY (Part 2)
hw3_improved_parser.sh / .py - improved PCKY (Part 4)
hw3_pcky.py - shared library: PCFG reader + PCKY
hw3_run.sh - runs everything end to end (Part 5)
hw3_trained.pcfg, parses_base.out/.eval, parses_improved.out/.eval - outputs

hw3_run.sh needs the gold parses for evalb, which is not one of its arguments, so it defaults to $D/parses.gold (GOLD=... overrides it) and finds evalb in /dropbox/26-27/571F/hw3/tools (EVALB_DIR=... overrides it). It uses COLLINS.prm and falls back to Collins.prm.

I improved the PARSER (not the induction), so there is no hw3_improved.pcfg and the <input_PCFG> argument of hw3_run.sh is accepted but ignored; the improved parser re-uses hw3_trained.pcfg. The shell scripts call the course Python (/mnt/dropbox/26-27/571F/envs/bin/python) and only the standard library is used.

2. PART 1 PCFG INDUCTION (hw3_topcfg)
Since each treebank line is already in CNF, each non-terminal node contributes to one rule: binary (A -> B C) or lexical (A -> 'word') for pre-terminals. I read the trees with a small tokenizer of my own, count Count(A -> beta) and Count(A), and then estimated P(A -> beta) = Count(A -> beta) / Count(A) (maximum likelihood with no smoothing and probabilities of each LHS sum to 1). The output is in the NLTK format and on parses.train: 856 rules over 95 non-terminals from 514 trees, start symbol TOP.

Design choices:

- The start symbol is the most frequent root label (TOP) and its rules are written first, since the NLTK convention (and my parser) takes the first rule's LHS as the start symbol.
- The probabilities are printed with 6 decimals instead of 5 so that rare rules cannot round to 0.00000.
- The terminals are quoted ('word'), with double quotes if the word has an apostrophe.
- The labels like NP_NNP and VP_PRIME are ordinary non-terminals and the gold trees keep them, so I don’t change them.

3. PART 2 PROBABILISTIC CKY (hw3_parser)
I implemented the PCKY myself (the NLTK is not used for parsing). The chart cell [i][j] maps each non-terminal A to (best log-probability of A over i..j, backpointer). I score log P(A->B C) + score(B) + score(C), keep only the best score per A, and revise the backpointer when a better derivation appears for each span and split point k, for each B in [i][k], C in [k][j] and rule A -> B C. The best parse is read off the backpointers from the start symbol in cell [0][n].

- Log space: scores are summed and not multiplied, which prevents underflow.

- The rules are indexed by (B, C) -> [(A, logp)], so each pair of child labels only looks at rules that can apply.
- Unary rules: a TOP -> S rule is unary, so after each cell is filled I run a unary closure (relax A -> B until no score improves but it terminates because log-probabilities are <= 0).
- Unparseable input: if a word has no tag or no start symbol spans the sentence, the parser writes a blank line, so the output always has one line per input sentence.
- Output: one bracketed parse per line, original words at the leaves.

Baseline limitation: The exact-match lexical lookup so any sentence with an unseen word does not parse.The baseline parsed 51 / 55 test sentences and the four failures were "Arriving before four p.m .", "Which of those leaves either Wednesday or Thursday ?", "Traveling first class ." and "Show me Ground transportation in Denver during weekdays ." (each has a word which was not seen in training, or one only seen in a different case).

4. PART 3 EVALUATION

The hw3_run.sh runs evalb (-p COLLINS.prm) on gold and hypothesis and compares them for both systems. But since evalb skips blank lines (unparsed sentences), so the baseline's score covers only the 51 sentences that it parsed.

5. PART 4 IMPROVEMENT: COVERAGE VIA BETTER OOV HANDLING

Dimension: Coverage. The only thing that changed is the lexical lookup in the parser and the CKY and the grammar are untouched. For a word w:

(1) w seen in training -> its training tags/probabilities (as in the baseline);

(2) else, if a case variant was seen ('Arriving' vs 'arriving') -> that word's tags and probabilities;

(3) else (truly OOV) -> propose every open-class tag with the same probability (1.0), and let the grammar decide which survives.

The "open-class" is estimated from the PCFG alone: a pre-terminal is open-class if it rewrites to >= 10 distinct words (MIN_DISTINCT). The threshold was chosen in advance, and not tuned on the test sentences.

Error analysis of the four newly parsed sentences (vs. gold):

- Exact match: "Arriving before four p.m ." (VBG found via the case-insensitive match to "arriving") and "Show me Ground transportation in Denver during weekdays ." - 2 of 4.
- * "Traveling first class ." - gold tag VBG, my output was VB (and S_VP instead of FRAG_VP). VBG has only 8 word types in training, so it was never proposed.
- "Which of those leaves either Wednesday or Thursday ?" - gold tag CC for "either", my output was NNP (and SQ instead of SQ_VP). As CC has only 2 word types, it was excluded too.
- Both errors come from the hard cutoff excluding the correct tag.

Second iteration (negative result): I tried graded lexical probabilities estimated from the PCFG alone to fix this and a hapax-based P(unknown word | tag) (the counts are recovered from the smallest lexical probability of each tag, with a

floor for single-word tags like TO), optionally times a 3-letter suffix model so unseen "-ing" words will prefer VBG. I compared the variants on 51 sentences held out from the training treebank (every 10th tree and induction on the other 463) to avoid tuning on the test data:

	parsed	recall/precision	complete match	tagging acc
baseline	37/51	83.13*	75.68	98.42
threshold	43/51	81.59	69.77	96.97
hapax	43/51	81.59	69.77	96.69
hapax + suffix	43/51	80.87	67.44	96.42

(* baseline scored on the 37 sentences it parsed)
The results were scored over all 51 sentences with the unparsed ones counted as zero, the baseline is about 60 and the threshold version is about 69. The baseline's higher raw number reflects that it is scored only on the 37 easier sentences it could parse. All variants parse the same 43/51 held-out sentences, and the simple threshold version scored best.

The ordering was the same on the test set (F1 88.29 for threshold vs. 88.00 for hapax+suffix; complete match 65.45% vs 63.64%). So I submitted the threshold version. The experimental modes remain in hw3_improved_parser.py behind OOV_MODE=hapax|hapax_suffix. The remaining unparsed held-out sentences probably need rules never seen in 463 training trees because all variants parse the same sentences: unknown-word tagging has a low ceiling, and smoothing the grammar rules would be the next thing to try. A possible reason the suffix model hurts (untested): suffix counts from ~500 sentences are noisy and outvote the grammar's structural preferences.

6. RESULTS (test set: 55 sentences of sentences.txt)

	baseline	improved (threshold)
sentences parsed	51/55	55/55
labeled recall	88.52*	88.29
labeled precision	88.52*	88.29
F1	88.52*	88.29
complete match	66.67 (34/51)	65.45 (36/55)
tagging accuracy	99.08	98.70
runtime	~0.03 s	~0.03 s

*evalb skips the 4 unparsed sentences, so the baseline is scored on 51 only.

Discussion: the improved parser raised coverage from 51/55 to 55/55 and left the runtime unchanged. Its F1 (88.29) covers all 55 sentences, while the baseline's 88.52 covers only 51 and counting the baseline's 4 blank lines as zero would put it at roughly 82, so the improved parser is clearly better overall. The small drop in F1 and tagging accuracy (99.08 -> 98.70) comes from guessing tags for unseen words. The complete matches rose from 34/51 to 36/55, i.e. 2 of the 4 newly parsed sentences are exactly right.

7. PROBLEMS AND WHAT I LEARNED

- Underflow/speed: multiplying many small probabilities is a trap and log space fixed it, and indexing rules by (B, C) made the inner loop fast.
- Root symbol: the first rule must be the start symbol (TOP), so the induction script writes the root's rules first.
- Unary root rule: TOP -> S needed unary handling in the chart.
- evalb ignores unparsed sentences, so baseline and improved F1 are not directly comparable and I reported the parsed counts and complete matches.
- My second iteration did not help. The held-out experiment on training data caught this before I submitted the more complicated model, which taught me to decide between methods on held-out data rather than on the test set.
- An MLE PCFG without smoothing is brittle: one unseen word, or one unseen rule, loses the whole sentence, so coverage is the real limit here.